PROJECT ROADMAP

Banking Management System

A Python Desktop Application (Tkinter GUI + SQLite Database)

Academic / Semester Project — Brainware University

Project Manager / Backend & Database Developer / Documentation Lead	Ramdas Hembram
Frontend/UI Developer / Testing & QA	Aditto Rudra
Duration	10 Weeks
Technology Stack	Python, Tkinter, SQLite3

1. Project Overview

The Banking Management System is a desktop application that simulates core banking operations — customer account creation with an auto-generated account number, secure PIN-based login, cash deposit and withdrawal, fund transfer between accounts, transaction history logging, and an administrator panel for oversight and account management. The system is built using Python with a Tkinter graphical interface and an SQLite relational database for persistent storage.

This roadmap spans 10 weeks and is structured to balance workload evenly between the two team members while reflecting their assigned roles: Ramdas Hembram leads project management, backend logic, database design, and documentation; Aditto Rudra leads UI/UX design and implementation, along with testing and quality assurance.

Objectives

- Deliver a fully functional, secure, and user-friendly banking desktop application.
- Demonstrate sound software engineering practice: requirements, design, modular development, testing, and documentation.
- Produce a polished, modern Tkinter interface that elevates the project beyond a typical academic CLI-style submission.
- Maintain balanced contribution and clear accountability between both team members.

2. Team Roles & Responsibilities

Member	Primary Roles	Core Responsibilities
Ramdas Hembram	Project Manager, Backend Developer, Database Developer, Documentation Lead	Requirement analysis, task scheduling, SQLite schema design, backend business logic (accounts, transactions, authentication), admin panel logic, SRS/report writing, coordination between modules.
Aditto Rudra	Frontend/UI Developer, Testing & Quality Assurance	Wireframing, Tkinter screen design, UI styling/theme, navigation flow, input validation on UI, integration testing, bug tracking, test-case design, user acceptance testing.

3. Project Phases

The project is organized into seven phases mapped across a 10-week timeline:

- Phase 1: Initiation
- Phase 2: Planning & Requirement Analysis
- Phase 3: System & UI Design
- Phase 4: Development (Backend + Frontend, parallel tracks)
- Phase 5: Integration
- Phase 6: Testing & Quality Assurance
- Phase 7: Deployment, Documentation & Presentation

4. Week-by-Week Timeline & Milestones

Week	Phase	Focus / Milestone	Owner(s)
Week 1	Initiation	Project charter finalized; topic approval from faculty guide; roles confirmed; GitHub repo created	Both (Ramdas leads)
Week 2	Planning	Requirement gathering; SRS document drafted; feature list & scope locked; DB entity list drafted	Ramdas (SRS), Aditto (feature UX input)
Week 3	Design	ER diagram & SQLite schema finalized; wireframes for all screens; UI theme/style guide chosen	Ramdas (DB design), Aditto (wireframes/theme)
Week 4	Development	Milestone: Database layer complete; login/signup screens built (static)	Ramdas (DB + auth logic), Aditto (UI screens)
Week 5	Development	Account creation + auto account-number generation; Dashboard UI complete	Ramdas (backend), Aditto (frontend)
Week 6	Development	Deposit/Withdraw modules; corresponding UI forms + validation	Ramdas (logic), Aditto (UI + validation)
Week 7	Development	Milestone: Fund transfer module; transaction history module (backend + UI table view)	Ramdas (logic), Aditto (UI)
Week 8	Integration	Admin panel (view/search/freeze accounts, view all transactions); full module integration	Ramdas (admin backend), Aditto (admin UI)
Week 9	Testing & QA	Milestone: Unit + integration testing; bug fixing; UI polish and consistency pass	Aditto leads testing; Ramdas fixes backend bugs
Week 10	Deployment & Docs	Milestone: Final build packaged; final report, user manual, PPT prepared; rehearsal & submission	Both (Ramdas: report, Aditto: manual/demo)

5. Detailed Tasks per Phase

Phase 1 — Initiation (Week 1)

Task	Assigned To	Notes
Define project title, scope, and objectives	Ramdas Hembram	Aligned with faculty guidelines
Get topic approved by project guide	Ramdas Hembram	Formal sign-off needed before Planning
Set up GitHub repository, folder structure, README	Ramdas Hembram	Shared with Aditto as collaborator
Set up local Python environment (venv, Tkinter, sqlite3 check)	Aditto Rudra	Ensures both machines are dev-ready
Create shared communication channel & task board	Both	See Tools section

Phase 2 — Planning & Requirement Analysis (Week 2)

Task	Assigned To	Notes
Gather functional & non-functional requirements	Ramdas Hembram	Interviews with guide, banking-app research
Draft Software Requirement Specification (SRS)	Ramdas Hembram	Deliverable of this phase
List required screens & user flows for UI	Aditto Rudra	Feeds directly into wireframing
Define feature list & prioritize (MoSCoW)	Both	Locks scope before design starts
Draft initial entity list for database (Users, Accounts, Transactions, Admin)	Ramdas Hembram	Input to ER diagram

Phase 3 — System & UI Design (Week 3)

Task	Assigned To	Notes
Design ER diagram (Accounts, Users, Transactions, Admin tables)	Ramdas Hembram	Depends on entity list (Phase 2)
Finalize SQLite schema (tables, keys, constraints)	Ramdas Hembram	Depends on ER diagram
Design account-number auto-generation logic (e.g., prefix + sequence)	Ramdas Hembram	Documented before coding starts
Create low-fidelity wireframes for all screens (Login, Signup, Dashboard, Deposit/Withdraw, Transfer, History, Admin)	Aditto Rudra	Depends on screen list (Phase 2)
Choose UI theme: color palette, fonts, custom Tkinter styling (ttk themes/icons)	Aditto Rudra	Key input for the "great UI" goal
Review & approve design jointly	Both	Gate before Development starts

Phase 4 — Development (Weeks 4–7, parallel tracks)

Task	Assigned To	Notes
Implement SQLite database creation script & connection handler	Ramdas Hembram	Foundation for all backend modules
Build authentication module (PIN hashing, login/signup logic)	Ramdas Hembram	Security-critical; uses hashing (e.g., hashlib)
Build account creation logic + auto account-number generator	Ramdas Hembram	Depends on DB schema
Build deposit / withdraw / balance-update logic with validation	Ramdas Hembram	Depends on account module
Build fund transfer logic (atomic debit/credit transaction)	Ramdas Hembram	Depends on deposit/withdraw logic
Build transaction history logging & retrieval queries	Ramdas Hembram	Runs alongside every money-movement function
Build admin backend (view users, freeze/unfreeze, view all transactions)	Ramdas Hembram	Depends on core modules being stable
Build Login & Signup screens in Tkinter (styled per theme)	Aditto Rudra	Depends on wireframes
Build Dashboard screen (balance display, navigation menu)	Aditto Rudra	Depends on login screen + backend session data
Build Deposit/Withdraw forms with input validation & confirmation dialogs	Aditto Rudra	Depends on backend function signatures
Build Fund Transfer screen (recipient lookup, amount entry, confirmation)	Aditto Rudra	Depends on transfer backend
Build Transaction History screen (scrollable table / Treeview widget)	Aditto Rudra	Depends on history backend
Build Admin Panel UI (account search, freeze toggle, reports view)	Aditto Rudra	Depends on admin backend
Apply consistent styling, icons, hover effects, error/success banners across all screens	Aditto Rudra	Ongoing — the "great UI" polish pass

Phase 5 — Integration (Week 8)

Task	Assigned To	Notes
Connect all UI screens to backend functions end-to-end	Both	Joint session recommended
Resolve data-flow issues between UI and database layer	Ramdas Hembram	Backend-side fixes
Verify navigation flow matches wireframes; fix layout breaks	Aditto Rudra	UI-side fixes
Freeze feature set for testing (no new features after this point)	Both	Gate before Testing phase

Phase 6 — Testing & Quality Assurance (Week 9)

Task	Assigned To	Notes
Write test cases covering all modules (login, transactions, transfer, admin)	Aditto Rudra	Deliverable: Test Case document
Execute functional & UI testing; log defects	Aditto Rudra	Uses bug tracker (see Tools)
Perform edge-case testing (negative amounts, insufficient balance, wrong PIN attempts)	Aditto Rudra	Security & robustness focus
Fix backend bugs identified during testing	Ramdas Hembram	Depends on defect log
Fix UI bugs and refine visual consistency	Aditto Rudra	Final polish pass
Regression test after fixes	Both	Confirms stability before packaging

Phase 7 — Deployment, Documentation & Presentation (Week 10)

Task	Assigned To	Notes
Package final application (requirements.txt, run instructions, .exe optional via PyInstaller)	Ramdas Hembram	Deliverable: deployable build
Write final project report (SRS, design, implementation, testing, conclusion)	Ramdas Hembram	Deliverable: Final Report
Prepare user manual / admin manual	Aditto Rudra	Deliverable: User Manual
Prepare presentation slides & live demo script	Both	Deliverable: PPT + demo
Rehearse demo, prepare Q&A for viva/faculty review	Both	Final readiness check

6. Deliverables by Phase

Phase	Deliverable(s)
Initiation	Approved project topic; GitHub repository
Planning	Software Requirement Specification (SRS) document
Design	ER diagram; SQLite schema document; wireframes; UI style guide
Development	Working modules: Auth, Accounts, Deposit/Withdraw, Transfer, History, Admin (backend + UI)
Integration	Fully connected working application (feature-complete build)
Testing & QA	Test case document; defect log; test summary report
Deployment & Docs	Final packaged application; final project report; user manual; presentation slides

7. Key Task Dependencies

- SRS (Phase 2) must be approved before ER diagram and wireframes (Phase 3) begin.
- SQLite schema (Phase 3) must be finalized before any backend module coding (Phase 4) starts.
- Wireframes and UI theme (Phase 3) must be approved before Tkinter screen development (Phase 4) starts.
- Authentication module must be completed before Dashboard and session-dependent screens can be built.
- Deposit/Withdraw logic must exist before Fund Transfer logic can be implemented (transfer reuses debit/credit functions).
- Core modules (Accounts, Transactions) must be stable before the Admin Panel (which reads/manages them) is built.
- All modules must be integrated (Phase 5) before formal Testing (Phase 6) begins.
- Testing sign-off must occur before final packaging and documentation (Phase 7).

8. Risk Management

Risk	Likelihood / Impact	Mitigation Strategy
Scope creep — adding extra features mid-development (e.g., loans, interest calculators) delaying the core system	Medium / High	Lock feature scope at end of Planning phase (MoSCoW prioritization); treat extras as "nice-to-have" stretch goals only after core features are stable.
Data loss or corruption in SQLite (e.g., concurrent writes, incomplete transactions during transfer)	Medium / High	Use atomic transactions (BEGIN/COMMIT/ROLLBACK) for all money-movement operations; maintain regular backups of the .db file during development; test failure scenarios explicitly.
Uneven workload or schedule slippage due to exams/other commitments (common in academic projects)	High / Medium	Weekly check-in meetings with a shared task board; buffer built into Week 9–10; tasks broken into small, trackable units with clear owners per the table above.
UI not meeting the "great UI" expectation — looking like a default/plain Tkinter app	Medium / Medium	Dedicate explicit design time in Phase 3 (theme, wireframes); use ttk styled widgets, consistent color palette, icons, and Treeview for tables rather than default widgets; run a UI polish pass in Week 9.
Security gaps — PINs stored in plain text, weak login validation	Low / High	Hash PINs before storage (e.g., hashlib/sha256 with salt); enforce PIN format validation and limit failed login attempts; document security measures in the final report.

9. Tools & Resources

Purpose	Tool(s)
Version Control	Git & GitHub (shared repository, feature branches, pull requests)
Communication & Task Tracking	WhatsApp/Discord for daily communication; Trello or GitHub Projects (Kanban board) for task tracking
UI/UX Design & Wireframing	Figma or Canva for wireframes; Tkinter + ttk (styled widgets), ttkbootstrap or custom themes for the "great UI" look
Backend Development	Python 3.x, sqlite3 (standard library), hashlib (PIN hashing)
Database Management/Inspection	DB Browser for SQLite
Testing	Python unittest / pytest for backend logic; manual test-case checklist for UI/UX
Documentation	Microsoft Word / Google Docs for SRS and final report; draw.io or Lucidchart for ER diagrams
Presentation	Microsoft PowerPoint / Canva for slides
Packaging (optional)	PyInstaller to produce a standalone .exe for demonstration

10. Workload Balance Summary

Both members carry a comparable load across the 10 weeks: Ramdas Hembram's effort is concentrated in backend/database logic and documentation, which are heavier in Weeks 2–8 but taper off during Weeks 9–10 (backend bug-fixing only). Aditto Rudra's effort is concentrated in UI development through Weeks 4–8, then intensifies in Weeks 9–10 as the lead for testing, QA, and the user manual. This creates a natural handoff where each member is at peak effort during complementary windows, keeping the overall pace balanced.

Member	Weeks 1–3 (Foundation)	Weeks 4–8 (Build)	Weeks 9–10 (Finish)
Ramdas Hembram	High (SRS, schema, planning)	High (all backend modules, admin logic)	Medium (bug fixes, final report)
Aditto Rudra	Medium (wireframes, theme, env setup)	High (all UI screens, styling)	High (testing, QA, manual, demo)

11. Achieving the "Great UI" Requirement

- A dedicated design step (Phase 3, Week 3) precedes any coding — wireframes and a color/typography style guide are agreed upon first.
- Use of ttk styled widgets and a custom theme (via ttkbootstrap or manual ttk.Style configuration) instead of raw default Tkinter widgets.
- Treeview widgets for transaction history and admin tables instead of plain text listings, giving a spreadsheet-like, professional feel.
- Consistent iconography, spacing, and color-coded feedback (e.g., green for success, red for errors) across all screens.
- A dedicated UI polish pass scheduled in Week 9 alongside testing, ensuring visual consistency is treated as a deliverable, not an afterthought.

Prepared for academic submission — Brainware University.